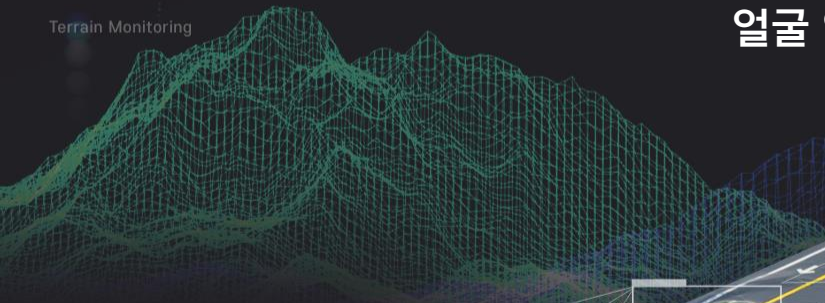


Image Recognition



Terrain Monitoring



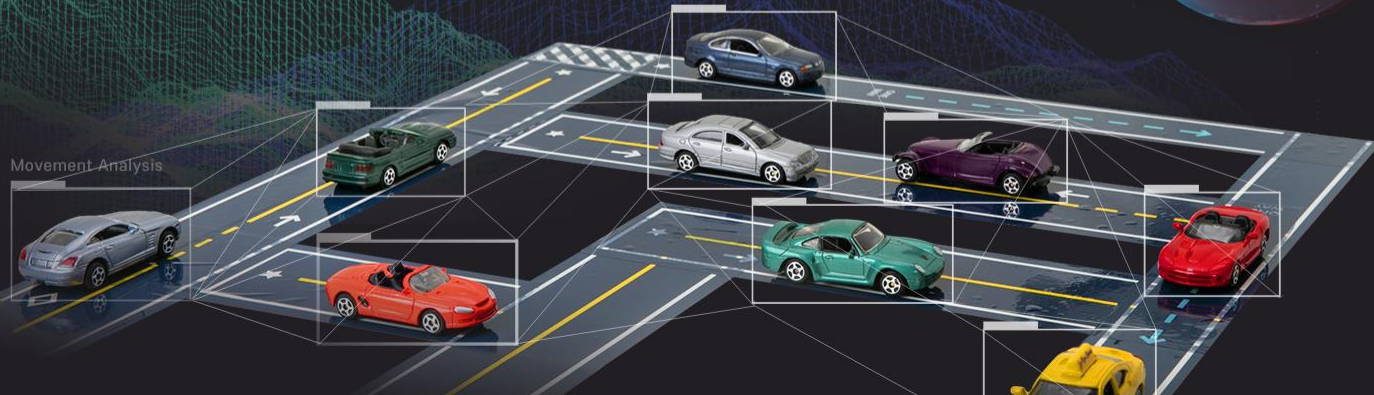
강원혁신플랫폼

컴퓨터 비전

얼굴 인식 알고리즘의 소개

Autonomous Driving

Movement Analysis

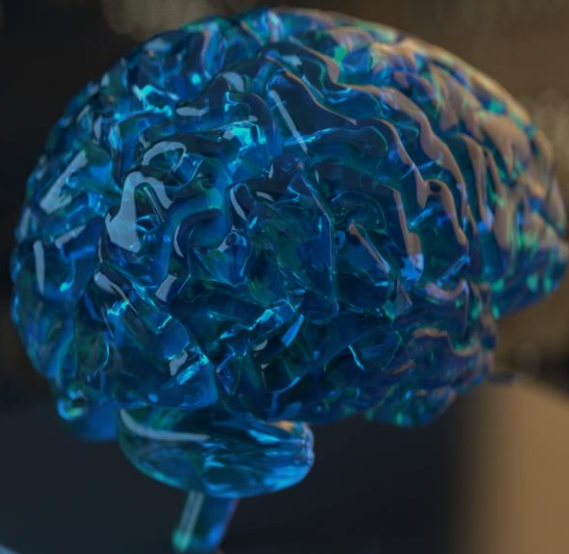


얼굴 인식



: face recognition

컴퓨터에게 사람의 얼굴을 보여주고
가지고 있는 여러 사진과 비교하여
가장 비슷한 사람을 맞추는 것임





사용자가 아닌 **개인 중심의 생체인증**이 대세



- 지문, 홍채, 정맥, 얼굴과 같은 **생체인증**은 이런 불편함을 해소하고 불특정 ‘사용자’가 아닌 ‘나’를 **인증**해 주는 **보안 수단**으로써 **각광**받고 있음



- 얼굴인증 기술은 교통, 주거, 온라인 등 **실생활의 다방면**에서 사용 가능하며, 그 **잠재력은 무궁무진함**

공항에서 활용

여권 없이 승객의 **생체정보**(얼굴)로 **신분증**을 **대체**해 체크인에서 탑승까지 있는 편리한 원아이디(One ID)





사이버 영역에서 활용

- 2020년 코로나19가 전 세계를 강타하면서 삶의 방식은 오프라인에서 **온라인**으로 순식간에 바뀜
- 사이버 영역에서의 보안은 익명의 사용자가 아닌 ‘내’가 접속했고 ‘내’가 계속 사용하고 있는지를 지속적으로 확인해야만 함
- 나를 확인하는 가장 확실한 방법은 얼굴인증임**

AI 기반의 얼굴인식 기술은 도시 관리(치안·교통 등), 자율주행, 무인점포, 헬스케어, 마케팅, 핀테크, 엔터테인먼트 등의 영역에서 광범위하게 사용됨

- 최근 국내에서는 얼굴인식 기술을 활용해 범죄 예방은 물론 치매 걸린 노약자나 아이들을 찾아내는 서비스를 선보임
- 중국에서는 수만 명이 운집한 콘서트장에서 수배자를 체포하는 등 사회 안전 영역에서 그 가치가 발휘됨



face_recognition



■ 파이썬(Python)을 위해 제공되는 심플한 안면 인식 API

- face_recognition은 얼굴 인식 라이브러리로 파이썬을 다룰 줄 알고, dlib 패키지를 설치하여 소스코드를 다운받으면 간단한 얼굴 인식 프로그램을 사용할 수 있음
- face_recognition을 이용해 이미지나 동영상 속에서 안면을 검출(detection)하고 검출된 얼굴들 중에서 특정한 인물을 표시하거나 가려내는 작업을 할 수 있음
- 이때 필요한 특정한 인물 1명의 사진 10장을 인터넷에서 다운로드 받음

여기서는 전현무 방송인의 사진 10장을 다운로드 받아 사용
(각자 좋아하는 방송인 사진을 다운로드)



face_recognition



- 인터넷에 있는 이미지를 그대로 다운 → 이미지의 사이즈, 선명도 등에 차이가 있을 수 있음
- 얼굴의 특정 값을 저장할 용도로 다운 → 되도록 정면 얼굴 사진을 선별해 다운 받음
- 다운로드 받은 각 사진 속 얼굴의 특성(encoding) 값과 특정한 인물의 이름을 매칭해 딕셔너리(dictionary) 타입으로 저장
- 새로운 이미지나 동영상에서 안면을 검출하고 검출된 얼굴들 중에서 특정한 인물이 있는지 가려내려 할 때 사용



이미지



68개의 얼굴 특징점



```
[ 0.097496, 0.045223, -0.128146, 0.032084,  
 0.192496, 0.145326, -0.428526, 0.133084,  
 0.027466, 0.085323, -0.227136, 0.042024,  
  ...  
 0.197696, 0.046223, -0.428146, 2.632084,  
 0.098426, 0.245623, -0.128346, 0.432024 ]
```

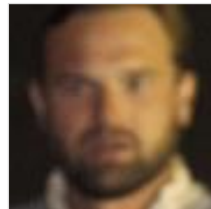
이미지에서 생성된 128가지 특징값

1 사진에서 얼굴을 찾는다.

- 사진에 등장하는 모든 얼굴을 찾을 수 있음



Input



Output

2 사진에 있는 얼굴의 특징을 찾고 조작한다.

- 사람의 눈, 코, 입, 턱의 위치와 윤곽을 잡아냄



Input



Output

3 사진 속 신원을 확인한다.

- 사진에서 누가 등장하였는지 인식



□ Input



Picture contains
“Joe Biden”

□ Output

face_recognition 라이브러리



- 딥러닝으로 구축된 **dlib** 라이브러리의 **얼굴 인식 기능**을 사용하여 **구축**되었으며 **이미지 파일**(영상) 속에서 **얼굴 인식** 및 **조작 가능**한 효율적인 **라이브러리**

라이브러리의 특징 정리

이미지에서 얼굴 찾기



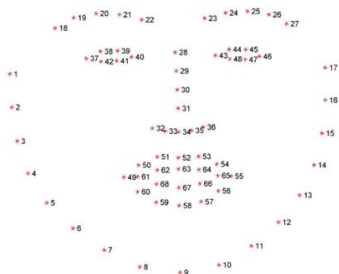
얼굴 특징 찾고 조작하기



신원 확인



이미지



68개의 얼굴 특징점



```
[ 0.097496, 0.045223, -0.128146, 0.032084,
  0.192496, 0.145326, -0.428526, 0.133084,
  0.027466, 0.085323, -0.227136, 0.042024,
  ...
  0.197696, 0.046223, -0.428146, 2.632084,
  0.098426, 0.245623, -0.128346, 0.432024 ]
```

이미지에서 생성된 128가지 특징값

■ face_recognition을 이용한 안면 인식의 특성



- 1 ▶ face_recognition 패키지는 **정확도면에서 한계가 있음**
- 2 ▶ 실전에서는 얼굴 특징값(face landmark)을 활용한 **이미지 전처리 과정(Alignment)**을 통해 **정확도를 높이고** 있음
- 3 ▶ **동양인들 끼리의 얼굴 인식은 잘되지 않음**

페이스북(Facebook)이 당신의 사진 속에서 친구들을 인식하는
신기한 기능을 이미 개발한 것을 알고 있을 것이다.

이전

페이스북에서는 사진을 클릭하고
이름을 입력해야만 친구들에 대한
태그를 달 수 있었음



현재

사진을 업로드 하자마자 페이스북은
마술처럼 사람들에게 대한 태그를 달아줌



사진에 사람들에게 대한
태그를 자동으로 달아줌

■ 얼굴 인식(face_recognition)은 여러 가지 관련된 문제들을 해결하는 과정임

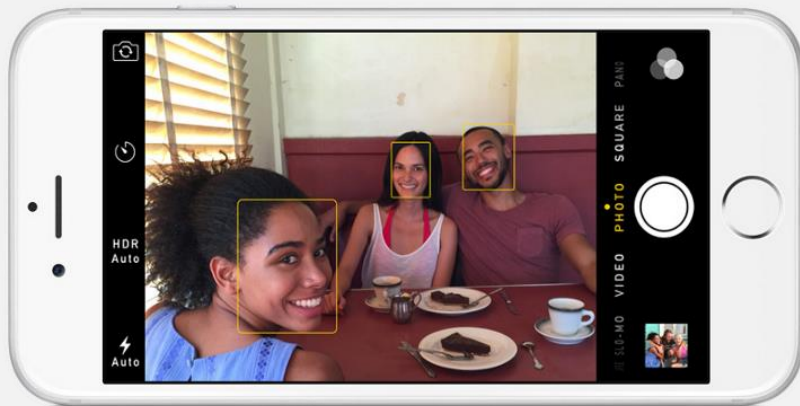
- ➡ 이중에서 얼굴의 위치 교정과 투영하는 단계는 아주 재미있음
- ➡ 인간으로서 당신의 두뇌는 얼굴 인식하는 모든 것을 자동으로 또는 즉시 수행하도록 연결되어 있음
- ➡ 사실 인간은 일상의 모든 사물의 얼굴 또는 표면을 발견하고 인식하는 일을 너무나도 잘 함

컴퓨터는 이러한 높은 수준의 일반화(적어도 아직까지는)를 할 수 없기 때문에
이 프로세스를 분리해서 각 단계별로 수행하는 방법을 가르쳐야 함

- 1 이미지에서 얼굴만 분리
- 2 얼굴 위치 교정(Posing)과 투영(Projection)
- 3 교정된 얼굴을 인코딩(encoding)
- 4 기존 데이터베이스와 비교하여 가장 비슷한 얼굴 찾기(인코딩에서 사람의 이름 찾기)

1 이미지에서 얼굴만 분리

- 첫 번째 단계는 이미지에서 **얼굴 검출**(face detection)임

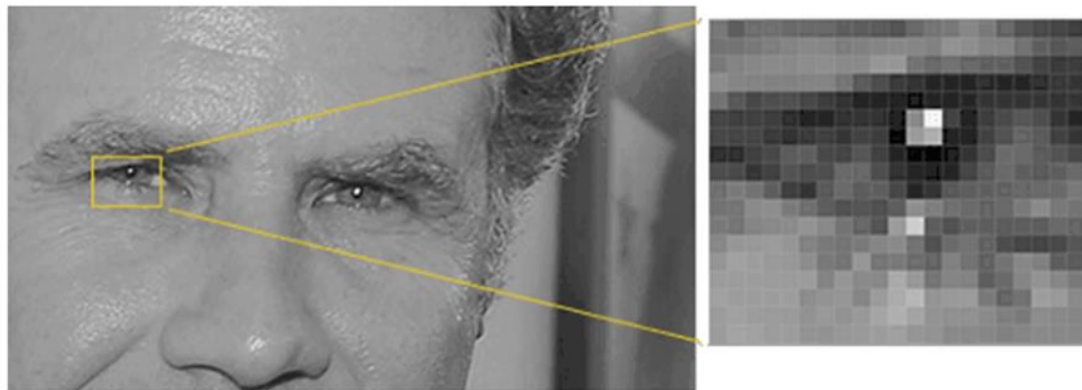


- 이미지에서 얼굴을 찾기 위해 **이미지**를 **흑백**으로 바꾸는 것부터 시작함
 - ➡ 왜냐하면, **얼굴**을 찾는데 **색상 데이터**는 **필요 없기 때문**



1 이미지에서 얼굴만 분리

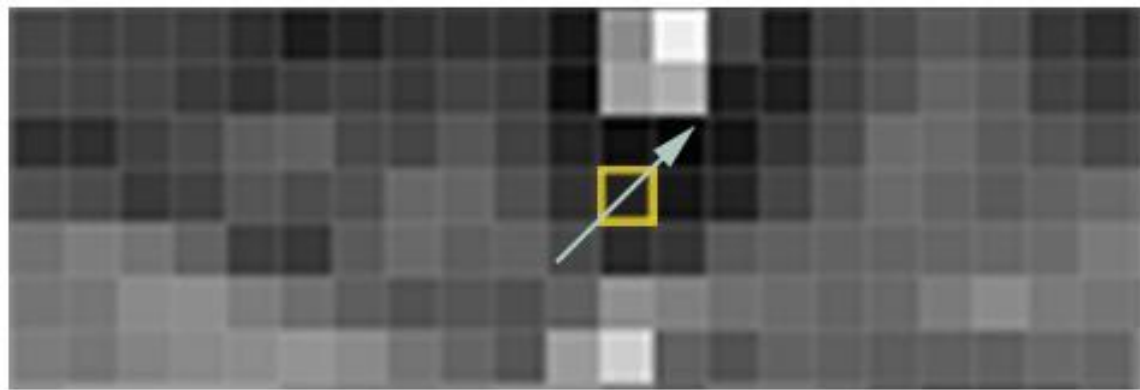
- 그런 다음 한번에 하나씩 **이미지**의 **모든 단일 픽셀**을 살펴보면 됨
- 🔄 모든 단일 픽셀에 대해 이를 직접 둘러싸고 있는 픽셀들 또한 살펴보면 될 것



- 🔄 여기서 중요한 목표는 해당 픽셀이 이를 직접 둘러싸고 있는 픽셀들과 비교해서 얼마나 어두운지 알아내는 것임

1 이미지에서 얼굴만 분리

- 그런 다음 이미지가 어두워지는 방향을 나타내는 화살표를 그려보자
 - ➡ 이미지의 모든 픽셀에 대해 이 프로세스를 반복하면 결국 모든 픽셀이 화살표로 바뀌게 됨
 - ➡ 이러한 화살표들을 그래디언트(gradients)라고 부름
 - ➡ 이를 통해 전체 이미지에서 밝은 부분으로부터 어두운 부분으로의 흐름을 알 수 있음

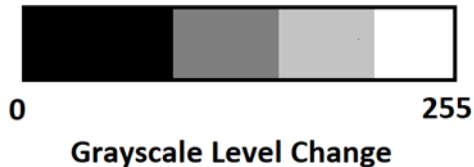


이 한 개의 픽셀과 이 픽셀에 인접한 픽셀들을 살펴 보면,
이미지는 오른쪽 상단으로 갈수록 어두워짐

1 이미지에서 얼굴만 분리

- 픽셀을 그래디언트(gradients)로 바꿔야 하는 정말 좋은 이유
 - ② 픽셀을 직접 분석하면, 동일한 사람의 정말 어두운 이미지와 정말 밝은 이미지는 전혀 다른 픽셀값을 갖게 될 것임
 - ② 밝기가 변하는 방향만 고려하면 정말 어두운 이미지와 정말 밝은 이미지에 대한 완전한 동일한 표현(representation)을 얻게 됨
 - ② 이렇게 하면 문제를 훨씬 더 쉽게 해결할 수 있음

$$8 \text{ bits color} = 2^8$$

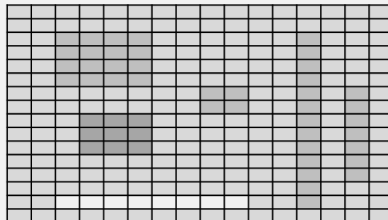


1 이미지에서 얼굴만 분리

- 여기서 **생각해볼** 문제는 모든 단일 픽셀에 대해 **그래디언트**(gradients)를 저장하면 너무 자세하다는 것
 - ② 이렇게 되면 결국 우리는 나무를 보고 **숲을 보지 못하게** 됨
 - ② 이미지의 기본 패턴을 알 수 있도록 **높은 수준**에서 밝음/어둠의 **기본 흐름**만을 보는 것이 더 좋음
- 이를 위해 이미지를 각각 **16x16(128) 픽셀**의 작은 **정사각형**들로 **분해**
 - ② 각 **정사각형**에서 얼마나 많은 **그래디언트**(gradients)가 **주요 방향**(윗쪽, 우상쪽, 오른쪽 등)을 얼마나 **가리키고** 있는지 세어 볼 것임
 - ② 그런 다음 이 **사각형**을 가장 강한 **화살표 방향**으로 **바꿀** 것임



이미지



16 x 16 픽셀의
정사각형

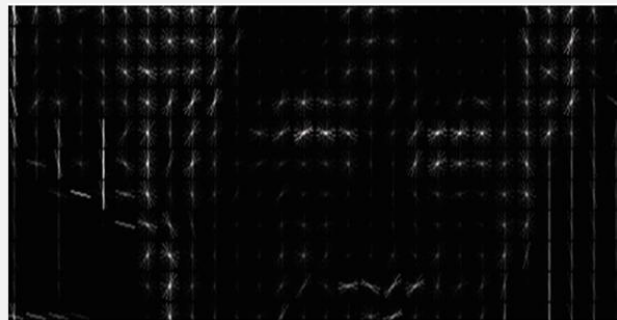
1 이미지에서 얼굴만 분리

- 결과적으로 원본 이미지를 얼굴의 기본 구조가 심플한 방법으로 표시되는 매우 간단한 표현(representation)으로 변환하게 됨

이미지에서 주요 특징들을 캡처한 HOG 표현으로 바꾼 것임



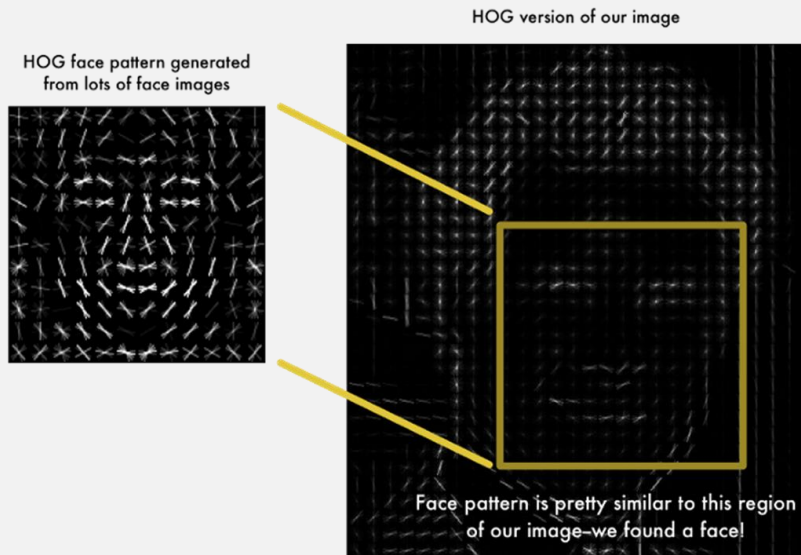
이미지



주요 특징들을 캡처한 HOG 표현

1 이미지에서 얼굴만 분리

- 그림과 같이 HOG 이미지에서 얼굴을 찾기 위해서 우리가 해야 할 일은 많은 훈련 얼굴 이미지로부터 추출된 잘 알려진 HOG 패턴과 가장 유사하게 보이는 부분을 이미지에서 찾는 것임



2 얼굴 위치 교정(Posing)과 투영(Projection)

■ 이미지에서 얼굴들만 분리해 보자

- ➡ 그러나, 이제 얼굴이 다른 방향을 보고 있으면 컴퓨터에게는 전혀 다르게 보이게 되는 새로운 문제를 해결해야 함
- ➡ 이를 해결하기 위해 각각의 사진을 비틀어 눈과 입술이 항상 표준 위치에 올 수 있도록 정규화 작업이 필요함
- ➡ 이렇게 하면 다음 단계들에서 얼굴을 좀 더 쉽게 비교 할 수 있음



인간은 두 이미지 모두 윌 페럴(Will Ferrell) 이라는 것을 쉽게 인식,
컴퓨터는 이 사진들을 완전히 다른 두 사람으로 보게 됨

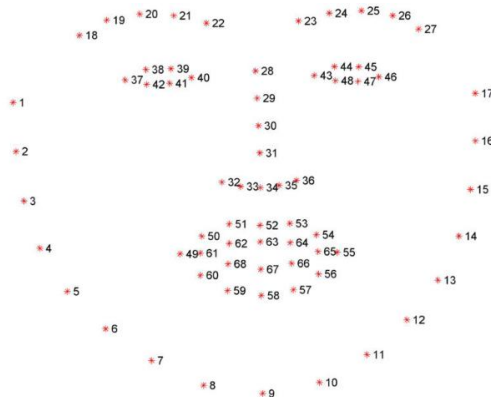


2 얼굴 위치 교정(Posing)과 투영(Projection)

- 얼굴 인식을 위한 정규화(눈과 입술이 항상 표준 위치)를 위해서
얼굴 특징값 추출(face landmark estimation) 이라고 하는 알고리즘을 사용

2014년 Vahid Kazemi & Josephine Sullivan이 개발

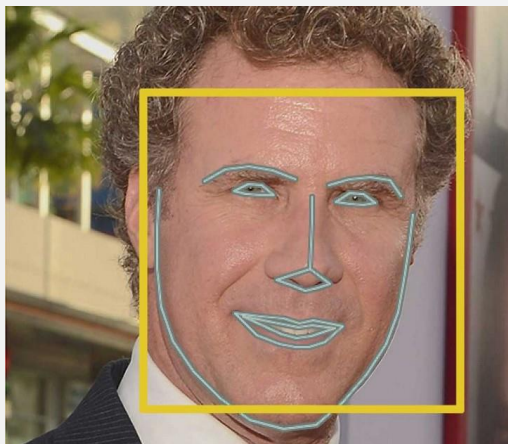
기본적인 아이디어는 모든 얼굴에 존재하는 68개의 랜드마크(landmarks)라 부르는 특정 포인트(턱의 상단, 눈 바깥의 가장자리, 눈썹 안쪽의 가장자리 등)를 찾아내는 것임



모든 얼굴에 표시할 68개의
랜드마크(landmarks)

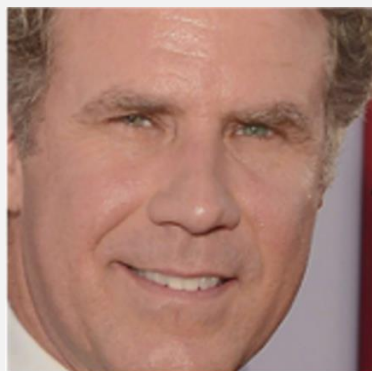
2 얼굴 위치 교정(Posing)과 투영(Projection)

- 이미지에서 68개의 얼굴 랜드마크(landmarks)를 표시한 결과
 - ↻ 눈과 입이 어디에 있는지 알게 됨
 - ↻ 이제 눈과 입이 최대한 가운데로 올 수 있도록 이미지를 간단하게 회전(rotate)하고, 크기를 조절(scale)하고, 비틀(shear)면 눈과 입술이 항상 표준 위치에 올 수 있도록 할 수 있음



2 얼굴 위치 교정(Posing)과 투영(Projection)

- 얼굴이 옆으로 돌아가 있어도 눈과 입을 이미지 상에서 대략 동일한 위치에 맞출 수 있음
- ➡ 이렇게 함으로써 다음 단계가 훨씬 더 정확해질 것임





3 교정된 얼굴을 인코딩(encoding)

■ 문제의 핵심인 실제로 얼굴을 구별하는 단계

- ② 얼굴 인식을 가장 간단하게 접근하는 방법은 2 단계(STEP 2)에서 찾아낸 모르는 얼굴을 이미 태그가 달린 사람들의 모든 사진과 직접 비교하는 것임
- ③ 모르는 얼굴과 매우 비슷하게 보이는 이미 태그가 달린 얼굴을 찾게 되면, 분명히 같은 사람일 것임

예) 각 귀의 크기, 눈 사이의 간격, 코의 길이 등을 측정해 볼 수 있음

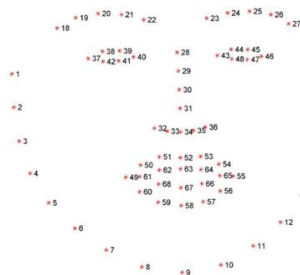


3 교정된 얼굴을 인코딩(encoding)

- 여기서 가장 신뢰할 수 있는 얼굴 측정 방법을 생각해 보자
- 최근에 연구자들은 컴퓨터가 수집할 측정값을 스스로 파악하도록 하는 것이 가장 정확한 접근 방법임을 알게 됨
- 즉 딥러닝은 얼굴의 어느 부분이 측정에 있어 중요한 것인지 파악하는 일을 인간보다 더 잘 수행함
- 사진에서 객체를 인식하도록 신경망을 훈련시키는 대신,
각 얼굴에 대해서 128개의 측정값을 생성하도록 훈련시킬 것임



이미지



68개의 얼굴 특징점



```
[ 0.097496, 0.045223, -0.128146, 0.032084,
  0.192496, 0.145326, -0.428526, 0.133084,
  0.027466, 0.085323, -0.227136, 0.042024,
  ...
  0.197696, 0.046223, -0.428146, 2.632084,
  0.098426, 0.245623, -0.128346, 0.432024 ]
```

이미지에서 생성된 128가지 특징값



3 교정된 얼굴을 인코딩(encoding)

■ 훈련 과정은 3개의 얼굴 이미지를 차례 대로 확인하는 것임

- 1 훈련용 아는 사람의 얼굴 사진 적재(load)
- 2 동일한 아는 사람의 다른 사진 적재
- 3 전혀 다른 사람의 사진 적재

↻ 1 2 이미지는 생성한 측정값이 서로 좀 더 가깝게 신경망을 조금 조정

↻ 2 3 이미지는 생성한 측정값이 서로 좀 더 멀어지도록 신경망을 조금 조정

3 채드 스미스(Chad Smith) 2 윌 페럴(Wil Ferrell) 1 윌 페럴(Wil Ferrell)



이미지에서 생성된 128가지 특징값



이미지에서 생성된 128가지 특징값



이미지에서 생성된 128가지 특징값

결과 비교

2 와 3 의 측정값은 좀 더 멀어지도록 신경망을 조금 조정



3 교정된 얼굴을 인코딩(encoding)

3 채드 스미스(Chad Smith)



이미지에서 생성된 128가지 특징값

2 윌 페럴(Wil Ferrell)



이미지에서 생성된 128가지 특징값

1 윌 페럴(Wil Ferrell)



이미지에서 생성된 128가지 특징값

결과 비교

2 와 3 의 측정값은 좀 더 멀어지도록 신경망을 조금 조정



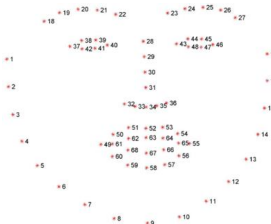
3 교정된 얼굴을 인코딩(encoding)

■ 얼굴 이미지 인코딩(encoding) 하기

- ⦿ 기계학습에서는 각 얼굴에 대한 128개의 측정값을 임베딩(embedding)이라고 부름
- ⦿ 임베딩(embedding)을 생성할 수 있도록 컨볼루션 신경망(Deep Convolution Neural Network, CNN)을 훈련시키는 프로세스는 많은 양의 데이터와 컴퓨터 자원을 필요로 함
- ⦿ 우리는 구글의 OpenFace 연구원들이 공개한 학습된 모델을 사용함
- » 따라서, 우리는 훈련된 신경망 모델을 사용해 얼굴 이미지로부터 각 얼굴에 대한 128개의 측정값을 얻으면 됨



이미지



68개의 얼굴 특징점



```
[ 0.097496, 0.045223, -0.128146, 0.032084,
  0.192496, 0.145326, -0.428526, 0.133084,
  0.027466, 0.085323, -0.227136, 0.042024,
  ...
  0.197696, 0.046223, -0.428146, 2.632084,
  0.098426, 0.245623, -0.128346, 0.432024 ]
```

이미지에서 생성된 128가지 특징값

3 교정된 얼굴을 인코딩(encoding)

- 전현무 방송인의 이미지에 대한 128개의 측정값이라 가정해 보자
- ➡ 그런데, 이러한 128개의 숫자들이 정확히 얼굴의 어느 부분을 측정한 것일까?

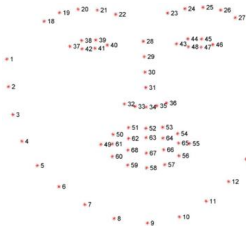
우리는 전혀 알 수가 없다

하지만, 전혀 문제가 되지 않는다

동일한 사람의 다른 두 개의 사진에 대해서 거의 같은 숫자들을 생성한다는 것이다



이미지



68개의 얼굴 특징점



[0.097496, 0.045223, -0.128146, 0.032084,
0.192496, 0.145326, -0.428526, 0.133084,
0.027466, 0.085323, -0.227136, 0.042024,
...
0.197696, 0.046223, -0.428146, 2.632084,
0.098426, 0.245623, -0.128346, 0.432024]

이미지에서 생성된 128가지 특징값

4 기존 데이터 베이스와 비교하여 가장 비슷한 얼굴 찾기(인코딩에서 사람의 이름 찾기)

- 이 마지막 단계는 전체 프로세스에서 가장 쉬운 단계임
- 🔄 우리가 해야 할 일은 새로운 이미지에 가장 근접한 측정값을 갖고 있는 사람을 데이터 베이스에서 찾아내는 것임

